

Immutable Publishing Inc.

Business Plan

"Verified Original Editions for the Digital Age"

Executive Summary

Immutable Publishing Inc. republishes public domain works with cryptographic verification, ensuring readers can prove they have the unaltered original text. In an era of silent content revisions, we offer permanence.

Mission: Preserve cultural heritage by publishing cryptographically verified editions of important public domain works.

Business Model: Low-cost publishing via Kindle Direct Publishing (KDP) and print-on-demand, differentiated by SHA-256 content verification.

Startup Cost: Under \$200 **Break-even:** ~50 book sales **Scalability:** Highly automated pipeline can publish 100+ titles/month

Problem Statement

The Silent Revision Problem

Publishers increasingly modify books without notice: - **Roald Dahl (2023):** Puffin removed "fat," "ugly," gender references - **Ian Fleming:** Bond novels edited for racial terms - **Agatha Christie:** Ongoing title and content changes - **Dr. Seuss:** Books pulled entirely from publication

Why This Matters

1. **Cultural Heritage Loss** - Original texts disappear from circulation
2. **No Verification Possible** - Readers can't prove they have the original
3. **Ebook Mutability** - Amazon can update Kindle books remotely (did so with *1984* in 2009)
4. **Academic Integrity** - Scholars need stable reference texts
5. **Collector Value** - No way to verify digital "first editions"

Market Gap

No publisher offers cryptographically verified editions. The technology is trivial; the will is absent because publishers benefit from silent revisions.

Solution

Cryptographically Verified Publishing

Each Immutable Publishing book includes:

1. **SHA-256 Hash** on copyright page
2. **Public Registry** at immutablepublishing.com
3. **Verification Tool** - Paste text, confirm authenticity
4. **"Verified Original" Branding** - Trust mark on cover

Example Copyright Page

IMMUTABLE PUBLISHING VERIFIED EDITION

Original text sourced from Project Gutenberg #11

First published: 1865 by Macmillan

Content Hash (SHA-256):

a7f8c3d2e1b4...

Verify at: immutablepublishing.com/verify/alice-wonderland

This text is cryptographically guaranteed to match Lewis Carroll's original 1865 publication.

Business Model

Revenue Streams

Channel	Price	Royalty	Net per Sale
Kindle eBook	\$2.99	70%	\$2.09
Kindle eBook	\$0.99	35%	\$0.35
Paperback	\$9.99	~\$2.50	\$2.50
Hardcover	\$19.99	~\$5.00	\$5.00

Pricing Strategy

- **eBooks:** \$2.99 (hits 70% royalty tier)
- **Paperbacks:** \$9.99-12.99 (competitive with mass market)
- **Premium Hardcovers:** \$19.99-24.99 (collectors)

Cost Structure

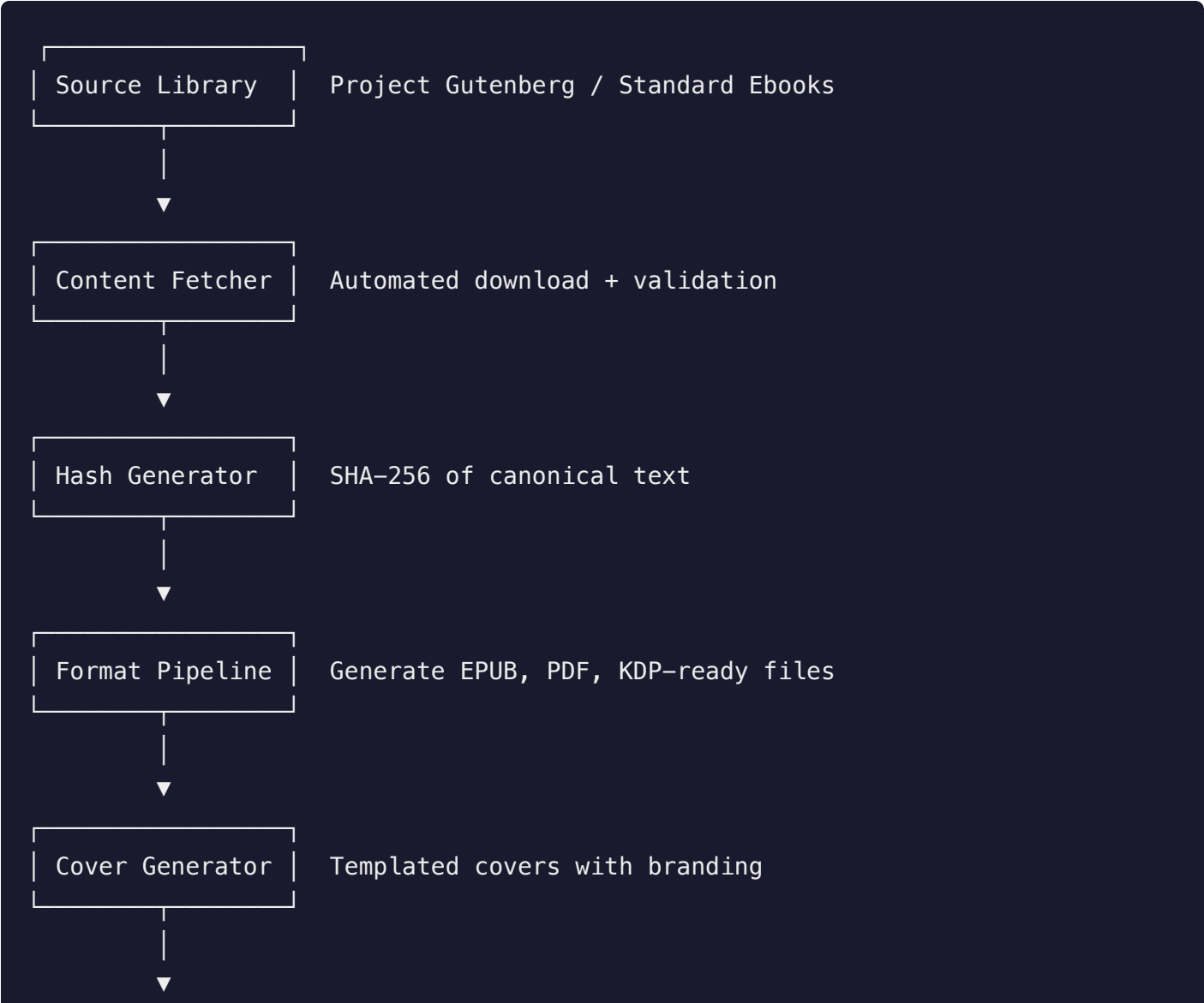
Item	Cost	Frequency
Domain	\$12	Annual
ISBNs (10-pack)	\$125	One-time
Cover templates	\$50	One-time
Hosting (GitHub Pages)	\$0	-

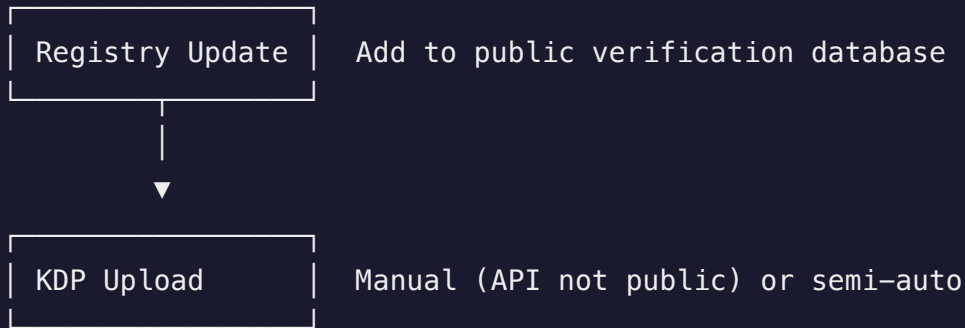
Item	Cost	Frequency
KDP account	\$0	-
Book formatting	\$0	Automated

Total Startup: \$187

Technical Architecture

Automated Publishing Pipeline





Automation Scripts

1. Book Fetcher (`fetch_book.py`)

```
#!/usr/bin/env python3
"""Fetch public domain book from Project Gutenberg"""

import requests
import hashlib
import json
from pathlib import Path

def fetch_gutenberg(book_id: int) -> dict:
    """Fetch book text and metadata from Project Gutenberg"""
    # Get plain text
    url = f"https://www.gutenberg.org/cache/epub/{book_id}/pg{book_id}.txt"
    response = requests.get(url)
    text = response.text

    # Strip Gutenberg header/footer (standardize)
    start = text.find("*** START OF")
    end = text.find("*** END OF")
    if start > 0 and end > 0:
        clean_text = text[text.find('\n', start)+1:end].strip()
    else:
        clean_text = text

    # Generate hash
    content_hash = hashlib.sha256(clean_text.encode('utf-8')).hexdigest()

    return {
        'gutenberg_id': book_id,
```

```

        'text': clean_text,
        'hash': content_hash,
        'source_url': url
    }

def save_book(book_data: dict, output_dir: Path):
    """Save book and metadata"""
    output_dir.mkdir(parents=True, exist_ok=True)

    # Save text
    (output_dir / 'content.txt').write_text(book_data['text'])

    # Save metadata
    meta = {k: v for k, v in book_data.items() if k != 'text'}
    (output_dir / 'metadata.json').write_text(json.dumps(meta, indent=2))

    print(f"Saved book {book_data['gutenberg_id']}")
    print(f"Hash: {book_data['hash']}")

if __name__ == '__main__':
    import sys
    book_id = int(sys.argv[1])
    data = fetch_gutenberg(book_id)
    save_book(data, Path(f"books/{book_id}"))

```

2. EPUB Generator (`generate_epub.py`)

```

#!/usr/bin/env python3
"""Generate KDP-ready EPUB from source text"""

from ebooklib import epub
import json
from pathlib import Path

def create_epub(book_dir: Path, title: str, author: str) -> Path:
    """Create EPUB with verification page"""

    meta = json.loads((book_dir / 'metadata.json').read_text())
    text = (book_dir / 'content.txt').read_text()

    book = epub.EpubBook()

```

```

book.set_identifier(f'immutable-{meta["gutenberg_id"]}')
book.set_title(title)
book.set_language('en')
book.add_author(author)

# Verification page
verify_html = f'''
<html><body>
<h1>Immutable Publishing Verified Edition</h1>
<p><strong>Content Hash (SHA-256):</strong></p>
<p style="font-family: monospace; word-break: break-all;">
{meta["hash"]}
</p>
<p>Verify at: immutablepublishing.com/verify</p>
<p>This text is cryptographically guaranteed to match the original publication
</body></html>
'''

verify_chapter = epub.EpubHtml(title='Verification', file_name='verify.xhtml')
verify_chapter.content = verify_html
book.add_item(verify_chapter)

# Main content (split into chapters if needed)
content_chapter = epub.EpubHtml(title=title, file_name='content.xhtml')
content_chapter.content = f'<html><body><pre>{text}</pre></body></html>'
book.add_item(content_chapter)

# Navigation
book.toc = [verify_chapter, content_chapter]
book.add_item(epub.EpubNcx())
book.add_item(epub.EpubNav())
book.spine = ['nav', verify_chapter, content_chapter]

output_path = book_dir / f'{title.lower().replace(" ", "-")}.epub'
epub.write_epub(str(output_path), book)

return output_path

```

3. Registry Manager (registry.py)

```
#!/usr/bin/env python3
"""Manage the public verification registry"""

import json
from pathlib import Path
from datetime import datetime

REGISTRY_FILE = Path('registry/books.json')

def add_to_registry(book_id: str, title: str, author: str,
                   content_hash: str, source: str):
    """Add book to public registry"""

    registry = json.loads(REGISTRY_FILE.read_text()) if REGISTRY_FILE.exists() else {}

    entry = {
        'id': book_id,
        'title': title,
        'author': author,
        'hash': content_hash,
        'source': source,
        'added': datetime.utcnow().isoformat(),
        'verify_url': f'https://immutablepublishing.com/verify/{book_id}'
    }

    # Update or add
    existing = next((b for b in registry['books'] if b['id'] == book_id), None)
    if existing:
        existing.update(entry)
    else:
        registry['books'].append(entry)

    REGISTRY_FILE.parent.mkdir(exist_ok=True)
    REGISTRY_FILE.write_text(json.dumps(registry, indent=2))

    return entry

def generate_static_site():
    """Generate static verification site for GitHub Pages"""
    registry = json.loads(REGISTRY_FILE.read_text())

    html = '''<!DOCTYPE html>
```

```

<html>
<head>
  <title>Immutable Publishing - Verify</title>
  <style>
    body { font-family: Georgia, serif; max-width: 800px; margin: 0 auto; padding: 10px; }
    .book { border-bottom: 1px solid #ccc; padding: 20px 0; }
    .hash { font-family: monospace; font-size: 12px; word-break: break-all; }
    input { width: 100%; padding: 10px; font-family: monospace; }
  </style>
</head>
<body>
  <h1>📖 Immutable Publishing</h1>
  <h2>Verification Registry</h2>

  <h3>Verify Your Copy</h3>
  <p>Paste your book's SHA-256 hash to verify authenticity:</p>
  <input type="text" id="verify-input" placeholder="Paste hash here...">
  <div id="verify-result"></div>

  <h3>Published Books</h3>
  ...

  for book in registry['books']:
    html += f'''
    <div class="book">
      <h4>{book['title']}</h4>
      <p><strong>Author:</strong> {book['author']}</p>
      <p><strong>Hash:</strong> <span class="hash">{book['hash']}</span></p>
    </div>
    ...

  html += '''
  <script>
  const registry = ''' + json.dumps(registry['books']) + ''';
  document.getElementById('verify-input').addEventListener('input', function(e) {
    const hash = e.target.value.trim().toLowerCase();
    const match = registry.find(b => b.hash.toLowerCase() === hash);
    document.getElementById('verify-result').innerHTML = match
      ? '<p style="color:green">✓ Verified: ' + match.title + ' by ' + match.author + ' with hash: '
      : hash.length === 64 ? '<p style="color:red">x Not found in registry</p>' : '<p></p>';
  });
  </script>
</body>

```

```
</html>'''
```

```
Path('docs/index.html').parent.mkdir(exist_ok=True)
Path('docs/index.html').write_text(html)
```

4. Cover Generator (generate_cover.py)

```
#!/usr/bin/env python3
"""Generate book covers using templates"""

from PIL import Image, ImageDraw, ImageFont
from pathlib import Path

def generate_cover(title: str, author: str, output_path: Path,
                  width: int = 1600, height: int = 2560):
    """Generate a simple, elegant cover"""

    # Create image
    img = Image.new('RGB', (width, height), '#1a1a2e')
    draw = ImageDraw.Draw(img)

    # Load fonts (fallback to default if not available)
    try:
        title_font = ImageFont.truetype('/System/Library/Fonts/Georgia.ttf', 120)
        author_font = ImageFont.truetype('/System/Library/Fonts/Georgia.ttf', 60)
        brand_font = ImageFont.truetype('/System/Library/Fonts/Helvetica.ttc', 40)
    except:
        title_font = ImageFont.load_default()
        author_font = title_font
        brand_font = title_font

    # Draw title
    draw.text((width//2, height//3), title, font=title_font,
              fill='#eee', anchor='mm')

    # Draw author
    draw.text((width//2, height//2), author, font=author_font,
              fill='#aaa', anchor='mm')

    # Draw brand
    draw.text((width//2, height - 200), 'IMMUTABLE PUBLISHING',
```

```

        font=brand_font, fill='#666', anchor='mm')
    draw.text((width//2, height - 140), '✓ VERIFIED ORIGINAL',
              font=brand_font, fill='#4caf50', anchor='mm')

    img.save(output_path, 'JPEG', quality=95)
    return output_path

```

5. Master Pipeline (`publish.sh`)

```

#!/bin/bash
# Full publishing pipeline

BOOK_ID=$1
TITLE=$2
AUTHOR=$3

echo "📖 Immutable Publishing Pipeline"
echo "====="

# Fetch source
echo "1. Fetching from Project Gutenberg..."
python3 fetch_book.py $BOOK_ID

# Generate formats
echo "2. Generating EPUB..."
python3 generate_epub.py books/$BOOK_ID "$TITLE" "$AUTHOR"

# Generate cover
echo "3. Generating cover..."
python3 generate_cover.py "$TITLE" "$AUTHOR" books/$BOOK_ID/cover.jpg

# Update registry
echo "4. Updating registry..."
python3 -c "from registry import add_to_registry; import json; \
    meta = json.load(open('books/$BOOK_ID/metadata.json')); \
    add_to_registry('$BOOK_ID', '$TITLE', '$AUTHOR', meta['hash'], 'gutenberg')"

# Generate site
echo "5. Regenerating verification site..."
python3 -c "from registry import generate_static_site; generate_static_site()"

```

```
echo ""  
echo "✅ Ready for KDP upload!"  
echo "    EPUB: books/$BOOK_ID/*.epub"  
echo "    Cover: books/$BOOK_ID/cover.jpg"  
echo ""  
echo "Manual step: Upload to kdp.amazon.com"
```

Go-to-Market Strategy

Phase 1: Launch (Month 1)

Goal: 20 cornerstone titles

1. **Classic Literature** (proven demand)
 2. Alice in Wonderland
 3. Pride and Prejudice
 4. Sherlock Holmes collection
 5. Dracula
 6. Frankenstein
7. **Philosophy** (academic market)
 8. Meditations (Marcus Aurelius)
 9. The Republic (Plato)
10. Nicomachean Ethics (Aristotle)
11. **Recently Controversial** (news hook)
12. Original Roald Dahl texts (pre-1928 where applicable)
13. Early Fleming (if any pre-1928)

Phase 2: Scale (Months 2-6)

- Expand to 100+ titles
- Automate cover generation with AI
- Build email list via verification site

- SEO for "original edition" searches

Phase 3: Premium (Months 6-12)

- Hardcover editions for collectors
 - "Library Edition" bundles
 - Academic partnerships
 - Audiobook verified editions
-

Marketing Channels

Free/Low-Cost Marketing

1. **SEO Content**
2. "Original vs revised [book name]"
3. "How to verify your ebook"
4. "Books that have been silently changed"
5. **Social Media**
6. Twitter/X threads on book alterations
7. Reddit: r/books, r/literature, r/privacy
8. Hacker News (tech angle)
9. **PR Hook**
10. "Publisher offers cryptographic proof of original text"
11. Pitch to tech and literary media
12. **Academic Outreach**
13. Literature departments need stable reference texts
14. Digital humanities angle

Paid (Optional)

- Amazon Ads: \$5-10/day test budget
- Target: "classic literature," "original edition," author names

Financial Projections

Year 1 Conservative Estimate

Metric	Value
Titles Published	50
Avg Sales/Title/Month	10
Monthly Sales	500
Avg Revenue/Sale	\$2.50
Monthly Revenue	\$1,250
Annual Revenue	\$15,000
Costs	\$500
Net Profit	\$14,500

Year 1 Optimistic (Viral Hit)

Metric	Value
Titles Published	100
Avg Sales/Title/Month	50
Monthly Sales	5,000

Metric	Value
Avg Revenue/Sale	\$2.50
Monthly Revenue	\$12,500
Annual Revenue	\$150,000

Break-Even Analysis

- Fixed costs: ~\$200
 - Variable cost per book: \$0 (digital)
 - At \$2.09 net per ebook: **96 sales to break even**
-

Risk Analysis

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Low sales volume	Medium	Medium	Focus on SEO, niche marketing
Amazon policy changes	Low	High	Diversify to other platforms
Copyright disputes	Low	Medium	Stick to clear public domain
Competition copies model	Medium	Low	First-mover advantage, brand
Technical verification bypass	Low	Low	Hash is proof, not DRM

Legal Considerations

Public Domain Rules

United States: - Works published before 1928: Public domain - Works by authors dead 70+ years: Check carefully - Government works: Public domain

Safe Sources: - Project Gutenberg (pre-vetted) - Standard Ebooks (pre-vetted) - Internet Archive (check dates)

Disclaimers

- "Original text edition" (not "definitive" or "authorized")
 - Clear source attribution
 - No copyright claim on public domain text
 - Copyright only on cover design, formatting
-

Implementation Roadmap

Week 1: Foundation

- ☐ Register domain: immutablepublishing.com
- ☐ Set up GitHub repo + Pages
- ☐ Create KDP account
- ☐ Design cover template
- ☐ Write automation scripts

Week 2: First Books

- ☐ Publish 5 pilot titles
- ☐ Test verification flow
- ☐ Gather initial reviews

Week 3-4: Scale

- ☐ Publish 15 more titles (20 total)
- ☐ Launch marketing push
- ☐ Set up analytics

Month 2+: Growth

- [] 10 new titles/week
- [] Content marketing
- [] Community building

Appendix A: Recommended First 20 Titles

#	Title	Author	Gutenberg ID
1	Alice's Adventures in Wonderland	Lewis Carroll	11
2	Pride and Prejudice	Jane Austen	1342
3	Frankenstein	Mary Shelley	84
4	Dracula	Bram Stoker	345
5	The Adventures of Sherlock Holmes	Arthur Conan Doyle	1661
6	Moby Dick	Herman Melville	2701
7	The Picture of Dorian Gray	Oscar Wilde	174
8	A Tale of Two Cities	Charles Dickens	98
9	The Great Gatsby	F. Scott Fitzgerald	64317
10	1984	George Orwell	(Not PD yet)
11	Meditations	Marcus Aurelius	2680
12	The Art of War	Sun Tzu	132
13	The Republic	Plato	1497

#	Title	Author	Gutenberg ID
14	Thus Spoke Zarathustra	Friedrich Nietzsche	1998
15	Crime and Punishment	Fyodor Dostoevsky	2554
16	War and Peace	Leo Tolstoy	2600
17	The Odyssey	Homer	1727
18	Don Quixote	Miguel de Cervantes	996
19	Grimm's Fairy Tales	Brothers Grimm	2591
20	The Prince	Niccolò Machiavelli	1232

Appendix B: Tool Requirements

```
# Python dependencies
pip install requests ebooklib pillow

# Optional: Better EPUB editing
pip install calibre # Or use Calibre GUI

# System tools (macOS)
brew install imagemagick # For cover manipulation
```

Appendix C: KDP Upload Checklist

For each book: - ☐ EPUB file (Kindle-compatible) - ☐ Cover image (2560x1600 minimum) - ☐ Title, subtitle, author - ☐ Description (include verification angle) - ☐ Keywords (7 allowed) - ☐

Categories (2 allowed) - [] Price (\$2.99 recommended) - [] Territories (worldwide)

Summary

Immutable Publishing Inc. fills a genuine market gap with minimal startup costs and high automation potential. The cryptographic verification angle is both technically simple and marketing-gold in the current climate of content revision concerns.

Next Steps: 1. Decide on domain/branding 2. Run the automation scripts on 5 pilot books 3. Launch, test, iterate

Business Plan v1.0 Prepared by Aineko 🐱 January 2026

Immutable Publishing - Blockchain & Public Good Addendum

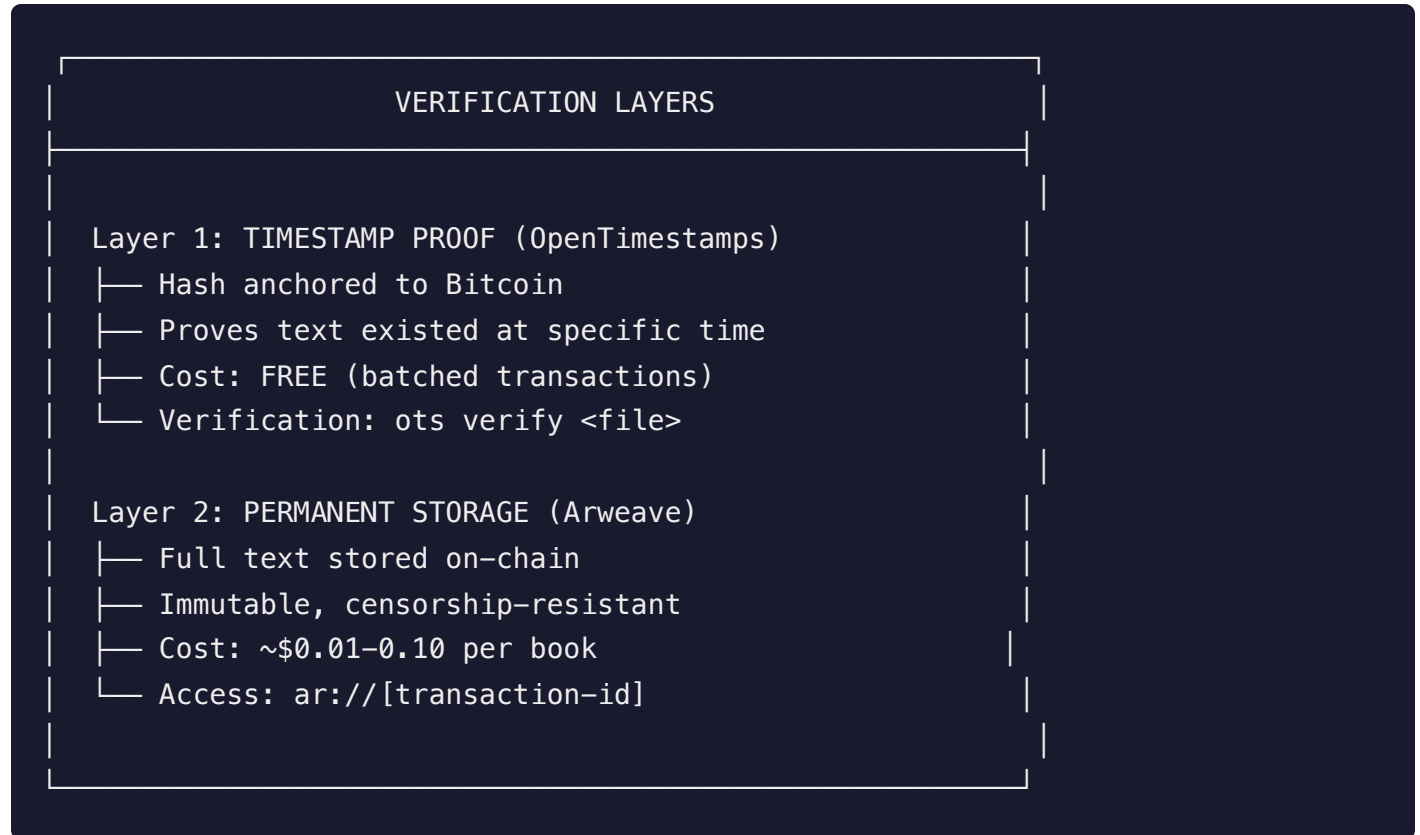
Philosophy Shift

Original model: Business selling verified books **New model:** Public good infrastructure for cultural preservation, sustainably funded

"The original texts of humanity's literary heritage should be freely available and cryptographically verifiable by anyone, forever."

Blockchain Architecture

Dual-Layer Verification



Why Two Layers?

Layer	Purpose	Survives
OpenTimestamps	Prove hash existed at time T	Website death, company death, server seizure
Arweave	Store actual content	Internet censorship, DMCA, company death

Together: **Cryptographic proof of what the text said, stored forever, verifiable by anyone**

OpenTimestamps Integration

What It Is

- Free, open-source Bitcoin timestamping
- Batches thousands of hashes into one BTC transaction
- Run by volunteers, no company to fail
- Used by Internet Archive, journalists, legal firms

How It Works

```
# Install
pip install opentimestamps-client

# Timestamp a file (creates .ots proof file)
ots stamp book.txt
# Uploads hash, returns immediately
# Proof confirmed when Bitcoin block is mined (~10 min – 2 hours)

# Verify a timestamp
ots verify book.txt.ots
# Output: "Success! Bitcoin block 871234 attests data existed as of 2026-01-17"
```

Integration Code

```
#!/usr/bin/env python3
"""OpenTimestamps integration for book verification"""

import subprocess
import hashlib
from pathlib import Path
from datetime import datetime

def timestamp_book(book_path: Path) -> Path:
    """Create Bitcoin-anchored timestamp for a book"""

    # Generate hash
    content = book_path.read_bytes()
    sha256 = hashlib.sha256(content).hexdigest()
```

```
# Create timestamp
result = subprocess.run(
    ['ots', 'stamp', str(book_path)],
    capture_output=True, text=True
)

ots_path = book_path.with_suffix(book_path.suffix + '.ots')

return {
    'book': str(book_path),
    'sha256': sha256,
    'ots_proof': str(ots_path),
    'timestamp': datetime.utcnow().isoformat(),
    'status': 'pending_confirmation' # Confirmed after Bitcoin block
}

def verify_timestamp(book_path: Path) -> dict:
    """Verify a book's timestamp against Bitcoin"""

    ots_path = book_path.with_suffix(book_path.suffix + '.ots')

    result = subprocess.run(
        ['ots', 'verify', str(ots_path)],
        capture_output=True, text=True
    )

    return {
        'verified': 'Success' in result.stdout,
        'output': result.stdout,
        'bitcoin_block': _extract_block(result.stdout)
    }

def _extract_block(output: str) -> str:
    """Extract Bitcoin block number from OTS output"""
    import re
    match = re.search(r'block (\d+)', output)
    return match.group(1) if match else None
```

Arweave Integration

What It Is

- Permanent, decentralized storage
- Pay once, stored forever (200+ year economic model)
- ~\$0.01-0.10 per book (text is small)
- Content-addressed: ar://[txid]

Setup

```
# Install Arweave bundler CLI
npm install -g @bundlr-network/client

# Fund your bundler account (accepts crypto or fiat)
bundlr fund 0.01 -c arweave

# Upload a file
bundlr upload book.txt -c arweave
# Returns: https://arweave.net/[transaction-id]
```

Integration Code

```
#!/usr/bin/env python3
"""Arweave permanent storage for books"""

import subprocess
import json
from pathlib import Path

def upload_to_arweave(book_path: Path, metadata: dict) -> dict:
    """Upload book to Arweave permanent storage"""

    # Add metadata tags
    tags = [
        f"-t", "Content-Type", "text/plain",
        f"-t", "App-Name", "ImmutablePublishing",
        f"-t", "Title", metadata.get('title', 'Unknown'),
        f"-t", "Author", metadata.get('author', 'Unknown'),
```

```
f"-t", "SHA256", metadata.get('hash', ''),
f"-t", "Source", metadata.get('source', 'Project Gutenberg'),
]

result = subprocess.run(
    ['bundlr', 'upload', str(book_path), '-c', 'arweave'] + tags,
    capture_output=True, text=True
)

# Parse transaction ID from output
# Output format: "Uploaded to https://arweave.net/[txid]"
txid = result.stdout.split('/')[-1].strip()

return {
    'arweave_txid': txid,
    'arweave_url': f'https://arweave.net/{txid}',
    'ar_protocol': f'ar://{txid}',
    'permanent': True
}

def verify_arweave(txid: str, expected_hash: str) -> dict:
    """Verify content on Arweave matches expected hash"""

    import requests
    import hashlib

    url = f'https://arweave.net/{txid}'
    response = requests.get(url)

    actual_hash = hashlib.sha256(response.content).hexdigest()

    return {
        'matches': actual_hash == expected_hash,
        'expected': expected_hash,
        'actual': actual_hash,
        'url': url
    }
```

Revised Business Model

Free Public Good Tier

Everything needed to verify and read original texts:

Asset	Location	Cost to User
Full text (TXT)	Arweave	Free
PDF (formatted)	Website + IPFS	Free
EPUB	Website + IPFS	Free
OTS proof file	Website + GitHub	Free
Verification tool	Open source	Free
Hash registry	Public API	Free

Funding: Donations, grants, institutional sponsors

Sustainable Revenue Tier

Convenience and physical goods:

Product	Price	Margin	Notes
Kindle delivery	\$0.99	35%	One-click convenience
Kindle (premium format)	\$2.99	70%	Enhanced formatting
Paperback	\$9.99-14.99	\$2-5	Print-on-demand
Hardcover	\$19.99-29.99	\$5-10	Collectors
Library bundle	\$99/year	100%	Institutional access
"Patron" tier	\$5-50/month	100%	Supporter donations

Funding Mix Target

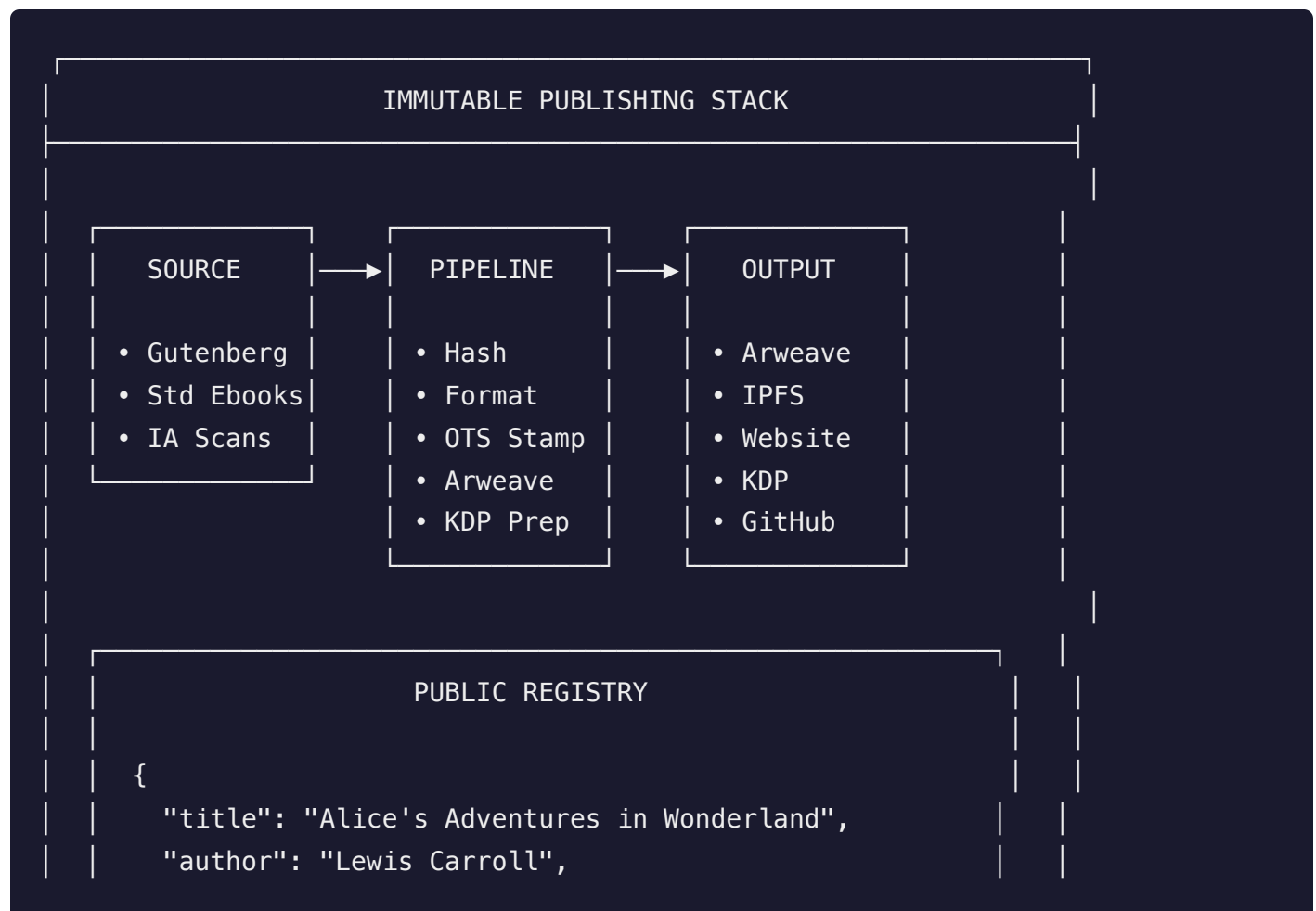
Year 1:

- |— Kindle/POD sales: 60%
- |— Donations: 30%
- |— Grants: 10%

Year 3 (mature):

- |— Kindle/POD sales: 40%
- |— Donations: 25%
- |— Grants: 15%
- |— Institutional: 15%
- |— Merchandise: 5%

Technical Architecture (Updated)



```

    "sha256": "a7f8c3d2e1b4...",
    "bitcoin_block": 871234,
    "bitcoin_timestamp": "2026-01-17T06:00:00Z",
    "arweave_txid": "Abc123...",
    "downloads": {
      "txt": "ar://Abc123...",
      "pdf": "https://immutablepub.com/dl/alice.pdf",
      "epub": "ipfs://Qm...",
      "ots": "https://immutablepub.com/proofs/alice.ots"
    }
  }
}

```

```

API: GET /api/v1/books/{id}
Bulk: GET /api/v1/registry.json

```

Updated Pipeline Script

```

#!/bin/bash
# publish.sh - Full pipeline with blockchain integration

BOOK_ID=$1
TITLE=$2
AUTHOR=$3

echo "📖 Immutable Publishing Pipeline (Blockchain Edition)"
echo "=====

# 1. Fetch source
echo "1. Fetching from Project Gutenberg..."
python3 fetch_book.py $BOOK_ID

# 2. Generate hash
echo "2. Computing SHA-256..."

```

```
HASH=$(sha256sum books/$BOOK_ID/content.txt | cut -d' ' -f1)
echo "    Hash: $HASH"

# 3. Bitcoin timestamp (OpenTimestamps)
echo "3. Creating Bitcoin timestamp..."
ots stamp books/$BOOK_ID/content.txt
echo "    Proof: books/$BOOK_ID/content.txt.ots"
echo "    (Will confirm in ~1-2 hours when Bitcoin block is mined)"

# 4. Permanent storage (Arweave)
echo "4. Uploading to Arweave..."
ARWEAVE_URL=$(bundlr upload books/$BOOK_ID/content.txt -c arweave \
    -t Content-Type text/plain \
    -t App-Name ImmutablePublishing \
    -t Title "$TITLE" \
    -t Author "$AUTHOR" \
    -t SHA256 "$HASH" | grep -o 'https://arweave.net/[^ ]*')
echo "    Permanent URL: $ARWEAVE_URL"

# 5. Generate formats
echo "5. Generating EPUB and PDF..."
python3 generate_epub.py books/$BOOK_ID "$TITLE" "$AUTHOR"
python3 generate_pdf.py books/$BOOK_ID "$TITLE" "$AUTHOR"

# 6. Generate cover
echo "6. Generating cover..."
python3 generate_cover.py "$TITLE" "$AUTHOR" books/$BOOK_ID/cover.jpg

# 7. Upload to IPFS (redundancy)
echo "7. Pinning to IPFS..."
IPFS_CID=$(ipfs add -q books/$BOOK_ID/content.txt)
echo "    IPFS: ipfs://$IPFS_CID"

# 8. Update registry
echo "8. Updating public registry..."
python3 registry.py add \
    --id "$BOOK_ID" \
    --title "$TITLE" \
    --author "$AUTHOR" \
    --hash "$HASH" \
    --arweave "$ARWEAVE_URL" \
    --ipfs "$IPFS_CID" \
    --ots "books/$BOOK_ID/content.txt.ots"
```

```
# 9. Generate static site
echo "9. Regenerating website..."
python3 registry.py generate-site

# 10. Deploy
echo "10. Deploying to GitHub Pages..."
cd docs && git add -A && git commit -m "Add $TITLE" && git push

echo ""
echo "✅ PUBLISHED!"
echo ""
echo "Free downloads:"
echo "  Arweave (permanent): $ARWEAVE_URL"
echo "  IPFS: ipfs://$IPFS_CID"
echo "  Website: https://immutablepublishing.com/books/$BOOK_ID"
echo ""
echo "Verification:"
echo "  Hash: $HASH"
echo "  OTS Proof: books/$BOOK_ID/content.txt.ots"
echo "  Bitcoin confirmation: pending (~1-2 hours)"
echo ""
echo "KDP upload (manual): books/$BOOK_ID/*.epub + cover.jpg"
```

Estimated Blockchain Costs

Per Book

Service	Cost	Notes
OpenTimestamps	\$0.00	Free forever
Arweave (~50KB text)	\$0.01	One-time, permanent
IPFS pinning	\$0.00	Free via Pinata free tier

Service	Cost	Notes
Total	~\$0.01	Per book, forever

At Scale (1000 books)

Item	Cost
Arweave storage	\$10
Domain	\$12/year
Hosting	\$0 (GitHub Pages)
Total Year 1	~\$22

Governance & Legal Structure

Recommended: Non-Profit (501c3) or Public Benefit Corp

Why: - Aligns with public good mission - Eligible for grants (NEH, Mellon, Sloan) - Tax-deductible donations - Credibility with institutions

Board composition: - Librarians / archivists - Digital humanities scholars - Open source advocates - Legal (copyright/public domain experts)

Alternative: DAO

For full decentralization: - Treasury controlled by token holders - Proposals for new books to publish - Community-driven curation

Pros: Censorship-resistant, community-owned **Cons:** Regulatory uncertainty, harder to get grants

Partnerships

Natural Allies

Organization	Collaboration
Internet Archive	Source texts, cross-promotion
Project Gutenberg	Official verification partner
Standard Ebooks	Share formatting, co-brand
EFF	Advocacy, legal support
Library of Congress	Credibility, grants
Universities	Research partnerships

Potential Funders

- **NEH (National Endowment for Humanities)** - Digital humanities grants
- **Mellon Foundation** - Library/archive funding
- **Sloan Foundation** - Tech + public interest
- **Knight Foundation** - Media/information integrity
- **Protocol Labs** - IPFS/Filecoin ecosystem grants
- **Arweave** - Permanent storage grants

Manifesto Draft

The Immutable Publishing Manifesto

The great works of human literature belong to everyone.

When a publisher silently changes Roald Dahl, when Amazon deletes 1984 from your Kindle, when sensitivity readers erase the past—we lose more than words. We lose our ability to understand where we came from.

We believe:

- 1. **Original texts should be freely available.** If it's in the public domain, it should be free to read.*
- 2. **Verification should be trustless.** You shouldn't have to trust us, or anyone. Verify against Bitcoin. Verify against Arweave. The math doesn't lie.*
- 3. **Preservation is a public good.** We exist to serve readers, scholars, and future generations—not shareholders.*
- 4. **Technology should serve culture.** Blockchains aren't just for speculation. They're tools for permanent, uncensorable truth.*

We publish books that cannot be unpublished. We preserve texts that cannot be altered. We verify truth that cannot be denied.

***Immutable Publishing** Verified. Permanent. Free.*

Summary

Adding blockchain transforms this from "business with a trust problem" to "public infrastructure for cultural preservation."

Cost increase: ~\$0.01 per book **Credibility increase:** Massive **Mission alignment:** Perfect

The paid tiers (Kindle, print) sustain the free public good. The blockchain ensures it survives regardless of what happens to the company.

This is how you build something that matters.

